



Centre for
Robotics



SLAM Course

08 - Factor Graphs

Dr Alejandro Fontán Villacampa
April 2026

Course Material Attribution

These slides were created by **Alejandro Fontan** as part of the **SLAM course**.

I have made every effort to acknowledge all sources from which slides, images, or videos were used. If I have made an error or omission, please let me know.

Special acknowledgments

- ❖ **Cyrill Stachniss** — *Photogrammetry and Robotics*. These slides draw heavily on Cyrill Stachniss's course material. That material in turn relied on the lecture notes by Wolfgang Förstner and the book Photogrammetric Computer Vision by Förstner & Wrobel. Parts of the robotics content are based on Probabilistic Robotics by Thrun, Burgard, and Fox.
- ❖ **Kaveh Fathian** — *Robotic Mapping and Localization*, Colorado School of Mines, Department of Computer Science. These slides draw heavily on the course material, which was itself compiled from several courses, lectures, and code repositories. Corresponding references are cited at the end of each lecture.
- ❖ **Antonio Torralba, Phillip Isola, and William T. Freeman** — *Foundations of Computer Vision*. Some images are sourced from this book.
- ❖ **L. Carlone, A. Kim, T. Barfoot, D. Cremers, and F. Dellaert** — *SLAM Handbook: From Localization and Mapping to Spatial Intelligence*, eds.. Cambridge University Press, 2026.

For Educators: university lecturers are welcome to use this material freely. If you adapt it, please retain all existing acknowledgments and add an acknowledgment to **Alejandro Fontan**. As a personal favour, I would appreciate an email notice if you use these slides.

Alejandro Fontan: alejandro.fontan@qut.edu.au



Factor Graphs



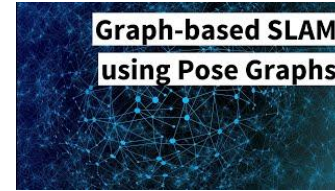
Factor Graph
5 Minutes with Cyrill
<https://youtu.be/uuiagGLFYa4>



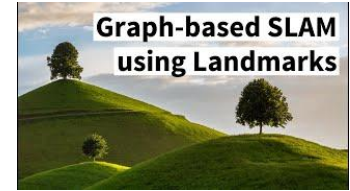
Tartan SLAM Series
Michael Kaess
<https://youtu.be/JmR2YpkLNt0>



Factor Graphs for Perception and Action
<https://youtu.be/-yCC7mpg4w>



Graph-based SLAM using Pose Graphs (Cyrill Stachniss)
<https://youtu.be/uHbRKvD8TWg>



Graph-based SLAM with Landmarks (Cyrill Stachniss)
<https://youtu.be/mZBdPgBtrCM>



SLAM Handbook

<https://github.com/SLAM-Handbook-contributors/slam-handbook-public-release>

<https://www.cs.cmu.edu/~kaess/pub/Dellaert17fnt.pdf>

<https://www.doc.ic.ac.uk/~ajd/Publications/Strasdat-H-2012-PhD-Thesis.pdf>

F. Dellaert, M. Kaess, and T. Barfoot, "Factor Graphs for SLAM," in *SLAM Handbook: From Localization and Mapping to Spatial Intelligence*, L. Carlone, A. Kim, T. Barfoot, D. Cremers, and F. Dellaert, Eds. Cambridge University Press, 2026.

F. Dellaert, M. Kaess, et al. 2017. "Factor Graphs for Robot Perception". *Foundations and Trends® in Robotics*, 6(1-2), 1–139.

H. Strasdat, "Monocular Exploration", in *Local accuracy and global consistency for efficient visual SLAM*, Doctoral dissertation, Department of Computing, Imperial College London, 2012



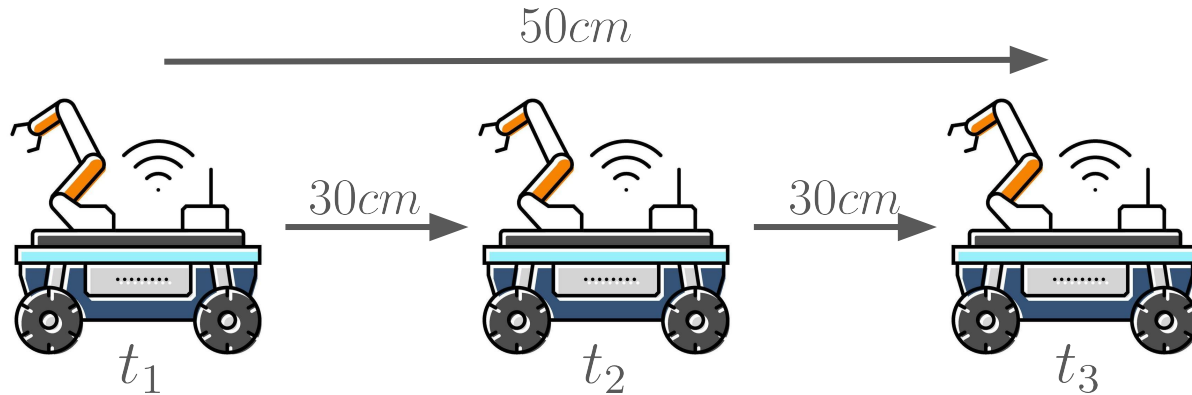
<https://github.com/alejandrofontan/SLAM-Course/>
https://github.com/ariarobotics/robotic-mapping/blob/main/lectures/Lec14_backend.pdf



Geometric Consistency Example

Geometric Consistency Example: Suppose we compute odometry estimates between $(t_1, t_2)(t_2, t_3)(t_1, t_3)$

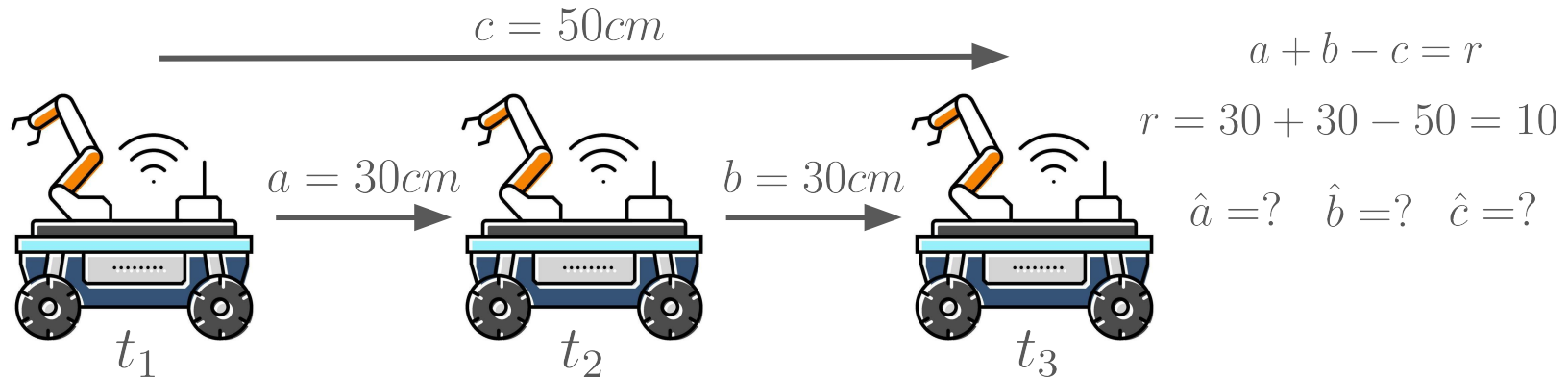
Estimates can be geometrically inconsistent (always are in practice!)



Geometric Consistency Example

Geometric Consistency Example: Suppose we compute odometry estimates between (t_1, t_2) (t_2, t_3) (t_1, t_3)

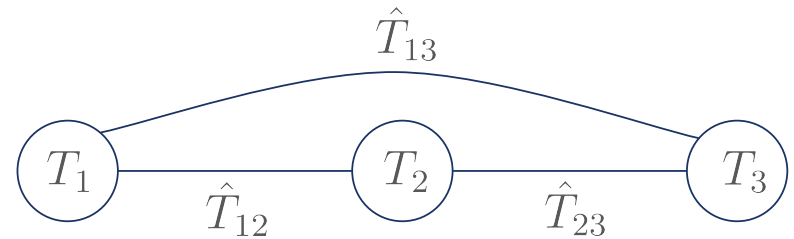
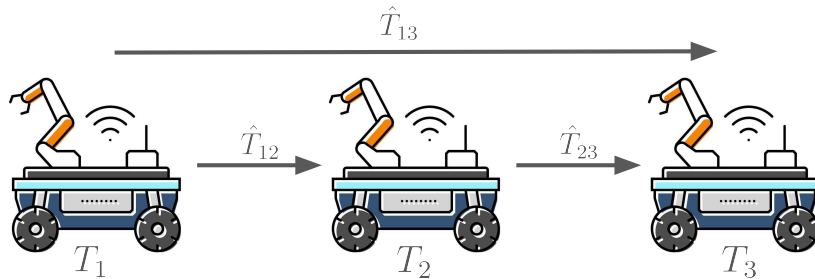
We can take average of the estimates to obtain consistent results



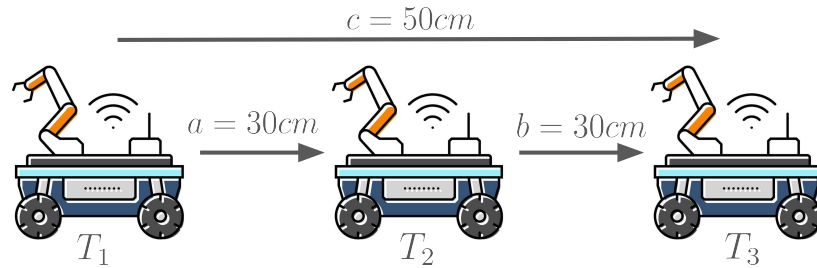
Factor Graph

Factor Graph: Mathematical framework for solving estimation/inference problems and obtain consistency

- Problem representation as a graph
- **Nodes/Vertices:** are optimization variable.
Can be of different types (continuous or discrete, poses or landmarks, etc.)
- **Factors/Edges:** represent relationship/constraints between variables (e.g. relative pose estimates, sensor measurements)

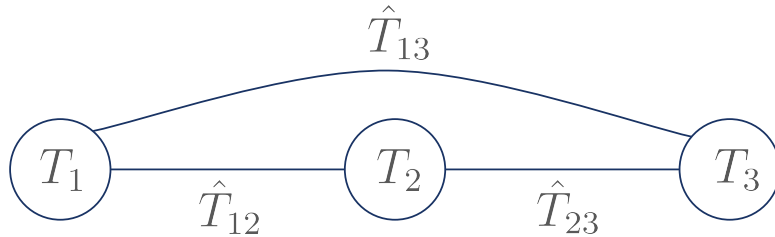


Factor Graph

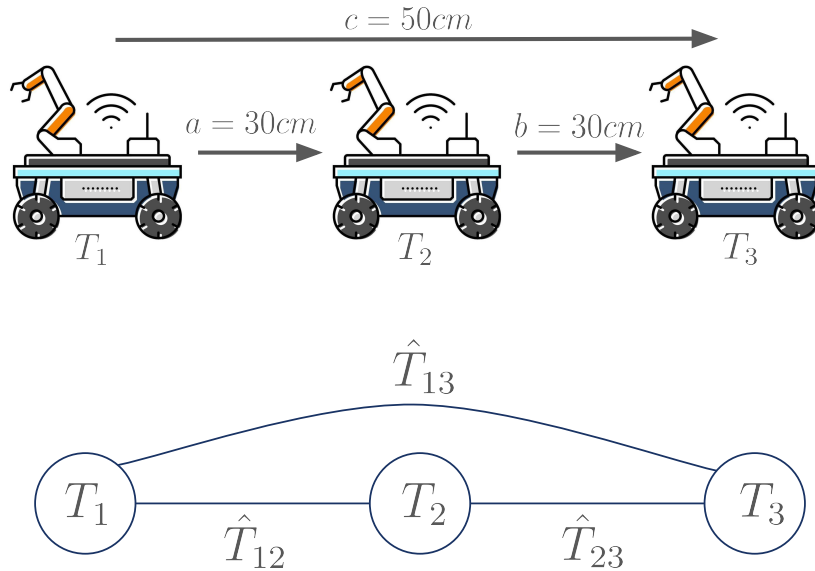


Pose/factor graph optimization:

$$\min_{T_t \in SE(3)} \sum_{(i,j) \in \mathcal{E}} \|T_i^{-1} T_j - \tilde{T}_{ij}\|_{\Sigma_{ij}}^2$$



Factor Graph

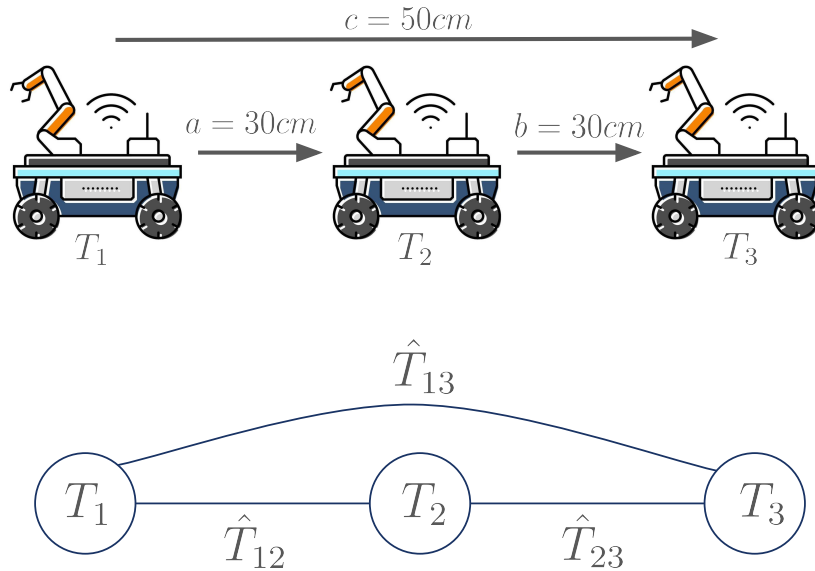


Pose/factor graph optimization:

$$\min_{T_t \in \text{SE}(3)} \sum_{(i,j) \in \mathcal{E}} \|T_i^{-1} T_j - \tilde{T}_{ij}\|_{\Sigma_{ij}}^2$$

λ	T_2	T_3
∞		
2		
1.5		
1		
0.5		
0		

Factor Graph



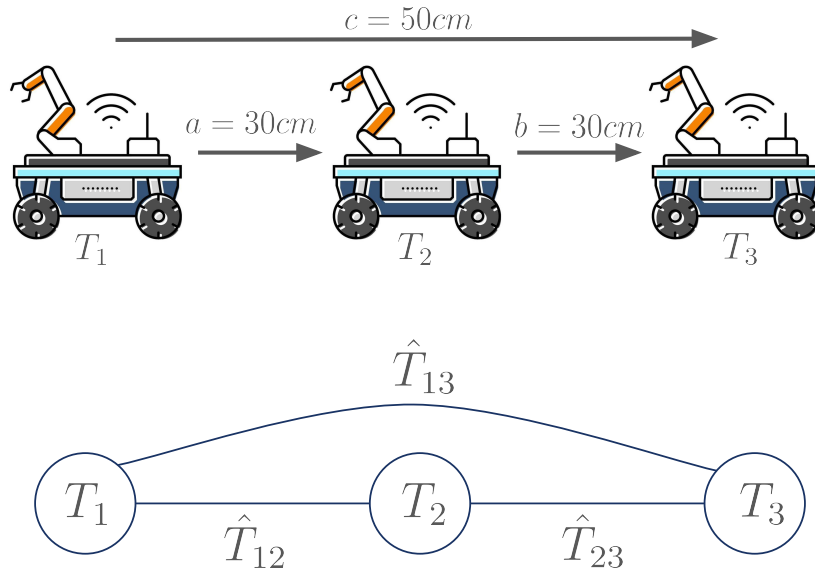
Pose/factor graph optimization:

$$\min_{T_t \in \text{SE}(3)} \sum_{(i,j) \in \mathcal{E}} \|T_i^{-1} T_j - \tilde{T}_{ij}\|_{\Sigma_{ij}}^2$$

λ	T_2	T_3
∞	30	60
2		
1.5		
1		
0.5		
0		



Factor Graph



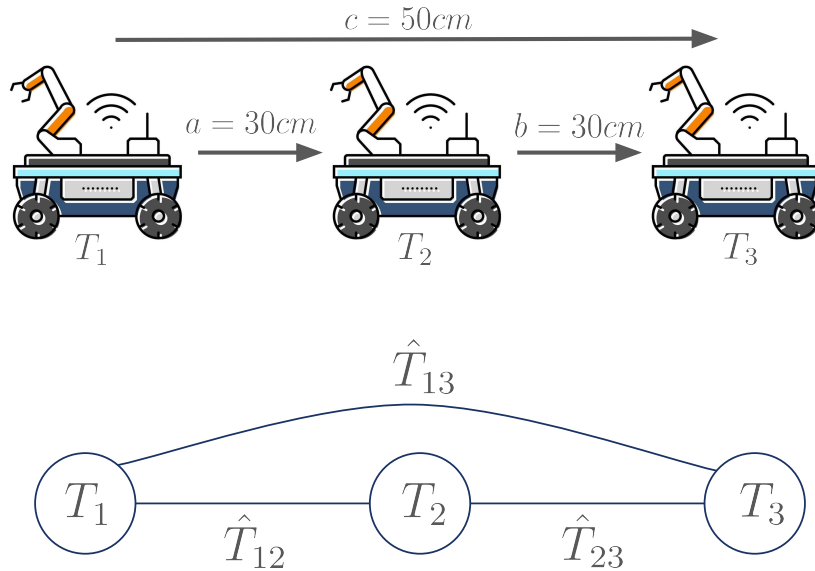
Pose/factor graph optimization:

$$\min_{T_t \in SE(3)} \sum_{(i,j) \in \mathcal{E}} \|T_i^{-1} T_j - \tilde{T}_{ij}\|_{\Sigma_{ij}}^2$$

λ	T_2	T_3
∞	30	60
2		
1.5		
1		
0.5		
0	25	50



Factor Graph

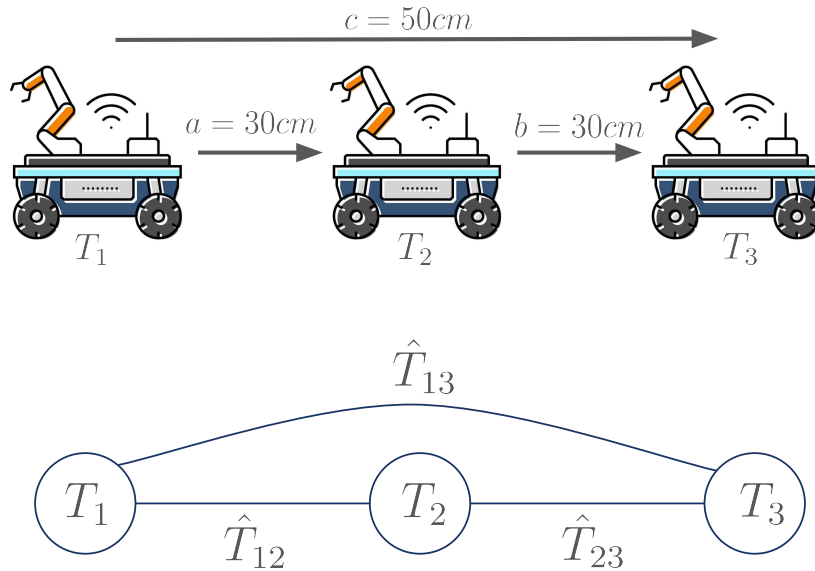


Pose/factor graph optimization:

$$\min_{T_t \in \text{SE}(3)} \sum_{(i,j) \in \mathcal{E}} \|T_i^{-1} T_j - \tilde{T}_{ij}\|_{\Sigma_{ij}}^2$$

λ	T_2	T_3
∞	30	60
2		
1.5		
1	26.67	53.33
0.5		
0	25	50

Factor Graph

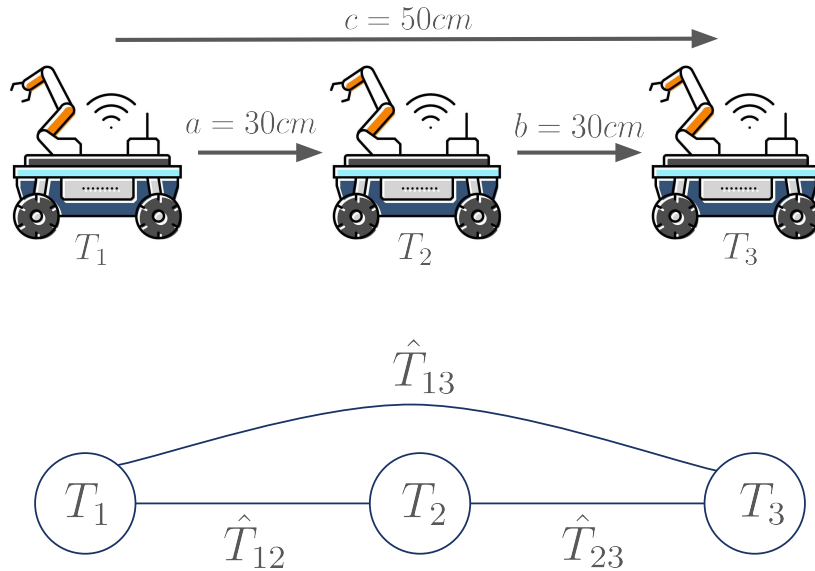


Pose/factor graph optimization:

$$\min_{T_t \in \text{SE}(3)} \sum_{(i,j) \in \mathcal{E}} \|T_i^{-1} T_j - \tilde{T}_{ij}\|_{\Sigma_{ij}}^2$$

λ	T_2	T_3
∞	30	60
2	27.5	55
1.5		
1	26.67	53.33
0.5		
0	25	50

Factor Graph



Pose/factor graph optimization:

$$\min_{T_t \in \text{SE}(3)} \sum_{(i,j) \in \mathcal{E}} \|T_i^{-1} T_j - \tilde{T}_{ij}\|_{\Sigma_{ij}}^2$$

λ	T_2	T_3
∞	30	60
2	27.5	55
1.5	27.14	54.28
1	26.67	53.33
0.5	26	52
0	25	50



What are Factor Graphs?

In this chapter we introduce **factor graphs** and establish the connection with **maximum a posteriori (MAP) inference** and **least squares for the case of Gaussian priors and Gaussian measurement noise**.

- ❖ We begin by **visualizing** SLAM problems using factor graphs,
- ❖ then show how **MAP inference** leads to **least squares optimization**.
- ❖ We discuss methods for solving both **linear and nonlinear optimization problem**,
- ❖ and then make the connection between **sparsity**, factor graphs, and Bayes nets more explicit.



Historical note

A smoothing approach to SLAM involves not just the most current robot location, SLAM but the **entire robot trajectory up to the current time**.

More generally, one can consider the full SLAM problem, i.e., the problem of **optimally estimating the entire set of sensor poses along with the parameters of all features in the environment**.

From a computational view, this optimization-based smoothing approach was recognized as beneficial since

- ❖ (a) in contrast to the filtering-based covariance or information matrices, which both become fully dense over time, **the information matrix associated with smoothing is and stays sparse**;
- ❖ (b) in typical mapping scenarios (i.e., not repeatedly traversing a small environment) this matrix is a much more **compact representation of the map covariance structure**.



A Toy Example: A simple SLAM Scenario

State

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{p}_3 \\ \ell_1 \\ \ell_2 \end{bmatrix}$$

**Probability density functions (PDFs)
over the unknown variables**

$$p(\mathbf{x})$$

Conditional density or posterior

$$p(\mathbf{x}|\mathbf{z})$$

1.1 Visualizing SLAM With Factor Graphs

23

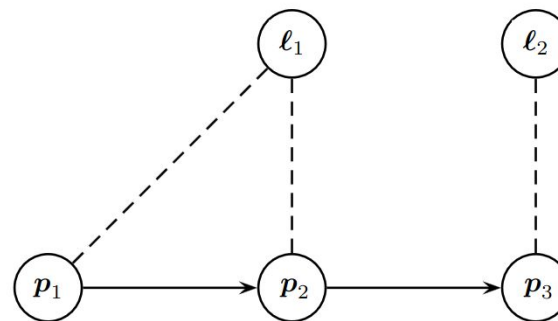


Figure 1.1 A toy simultaneous localization and mapping (SLAM) example with three robot poses and two landmarks. Above we schematically indicate the robot motion with arrows, while the dotted lines indicate bearing measurements.



Graphical Models: Bayesian Networks

24

Factor Graphs for SLAM

Bayesian networks consist of variables nodes, each associated with a prior or conditional probability density.

Convention: indicate *known* quantities as square nodes

Bayesian networks are excellent for modeling

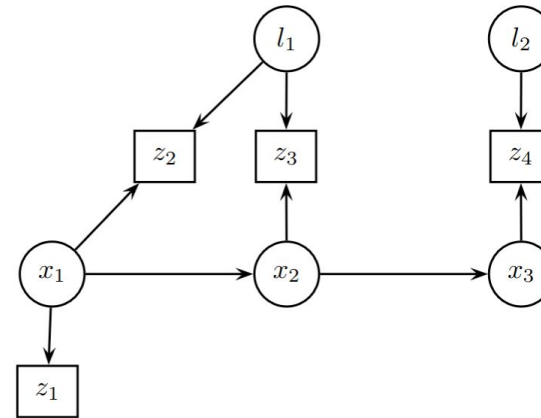


Figure 1.2 Bayesian network for the example in Figure 1.1, explicitly showing *known* measurements $z_1 \dots z_4$ as square nodes.



Graphical Models: Bayesian Networks

24

Factor Graphs for SLAM

The posterior can be re-written using Bayes' law:

$$\begin{aligned} p(\mathbf{x}|\mathbf{z}) &\propto p(\mathbf{p}_1)p(\mathbf{p}_2|\mathbf{p}_1)p(\mathbf{p}_3|\mathbf{p}_2) && \textbf{Robot motion} \\ &\times p(\ell_1)p(\ell_2) && \textbf{Landmark priors} \\ &\times p(\mathbf{z}_1|\mathbf{p}_1) && \textbf{First measurement} \\ &\times p(\mathbf{z}_2|\mathbf{p}_1, \ell_1)p(\mathbf{z}_3|\mathbf{p}_2, \ell_1)p(\mathbf{z}_4|\mathbf{p}_3, \ell_2) && \textbf{Bearing measurements} \end{aligned}$$

Each of the factors represents one piece of information about the unknowns.

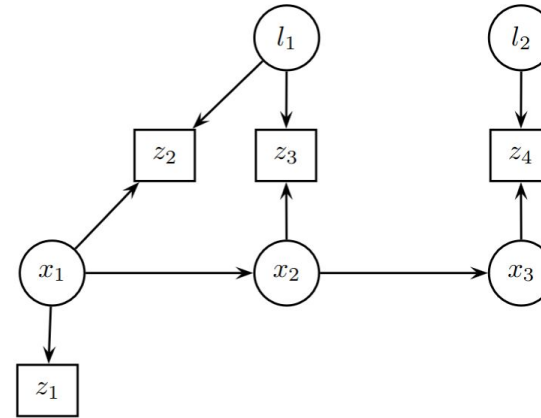


Figure 1.2 Bayesian network for the example in Figure 1.1, explicitly showing *known* measurements $\mathbf{z}_1 \dots \mathbf{z}_4$ as square nodes.

Graphical Models: Factor Graphs

Factor graphs are probabilistic graphical models that allow us to specify any density as a product of factors.

All unknown states $\mathbf{x}$, both poses and landmarks, have a node associated with them.

Measurements are not represented explicitly as they are known.

Small black nodes represent **factors**, and, importantly, **are connected to only those state variables of which they are a function**.

1.1 Visualizing SLAM With Factor Graphs

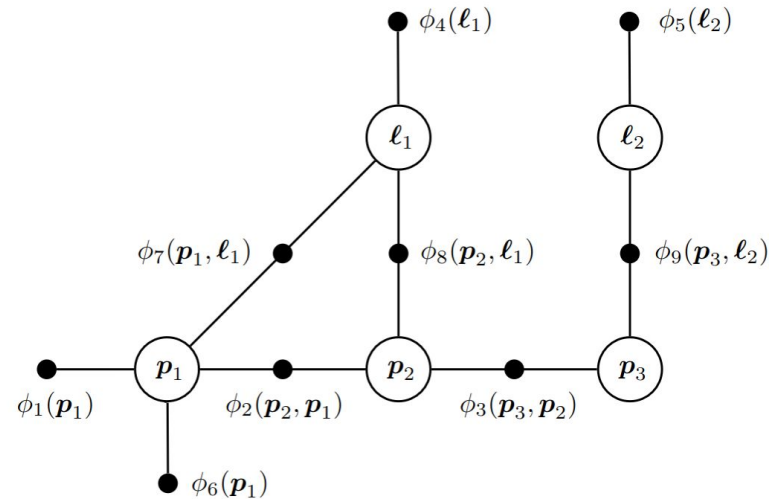


Figure 1.3 Factor graph resulting from the example in Figure 1.1.



Graphical Models: Factor Graphs

The factor values need only be proportional to the corresponding probability densities: any normalization constants that do not depend on the state variables may be omitted without consequence.

$$\begin{aligned}\phi(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \ell_1, \ell_2) &= \phi_1(\mathbf{p}_1) \phi_2(\mathbf{p}_2, \mathbf{p}_1) \phi_3(\mathbf{p}_3, \mathbf{p}_2) \\ &\times \phi_4(\ell_1) \phi_5(\ell_2) \\ &\times \phi_6(\mathbf{p}_1) \\ &\times \phi_7(\mathbf{p}_1, \ell_1) \phi_8(\mathbf{p}_2, \ell_1) \phi_9(\mathbf{p}_3, \ell_2)\end{aligned}$$

Although the measurement variables are not explicitly shown in the factor graph, those factors are implicitly conditioned on them.

1.1 Visualizing SLAM With Factor Graphs

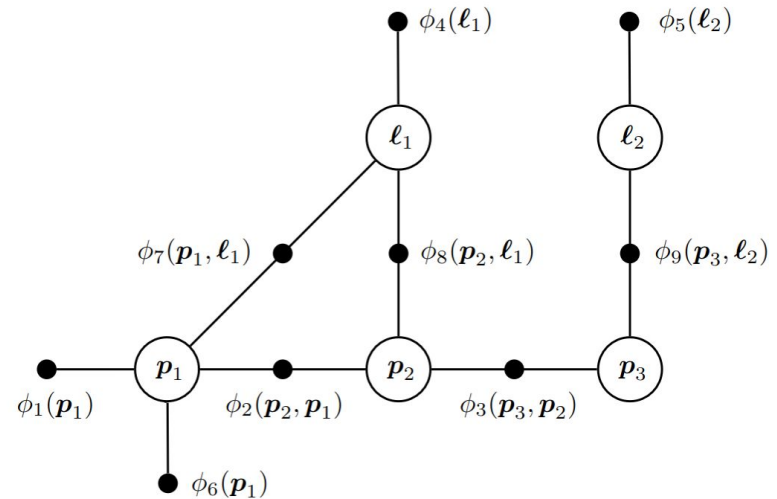


Figure 1.3 Factor graph resulting from the example in Figure 1.1.

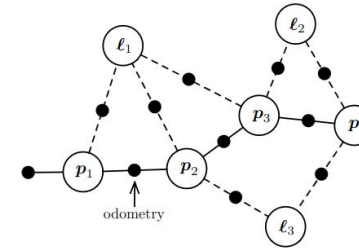


Factor Graphs as a Language

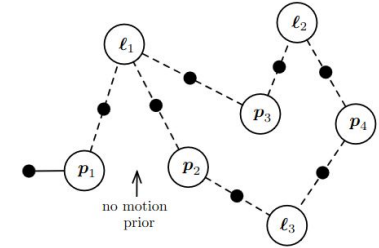
- ❖ **Landmark-based SLAM** has both pose and landmark variables; landmarks are measured from poses and there is some motion prior between poses typically based on odometry
- ❖ **Bundle Adjustment (BA)** is the same but without the motion prior
- ❖ **Pose-Graph Optimization (PGO)** does not have landmark variables but enjoys extra loop closure measurements between poses
- ❖ **Simultaneous Trajectory Estimation and Mapping (STEAM)** is similar to landmark-based SLAM, but now poses can be replaced by higher-order states and a smooth continuous-time motion prior is used

26

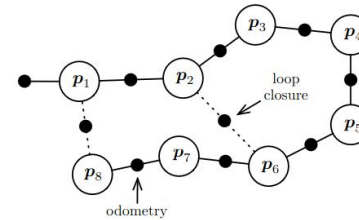
Factor Graphs for SLAM



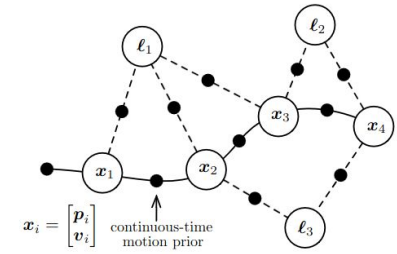
canonical landmark-based SLAM



bundle adjustment (BA)
(structure from motion)



pose-graph optimization (PGO)
(pose-graph SLAM)



simultaneous trajectory estimation
and mapping (STEAM)

Factor Graphs as a Language

Examining the factor graph reveals a great deal of structure by which we can gain insight into a particular instance of the SLAM problem:

- ❖ First, there are landmarks with a great deal of measurements, which we expect to be pinned down very well.
- ❖ Others have only a tenuous connection to the graph, and hence we expect them to be less well determined. For example, the lone landmark near the bottom right has only a single measurement associated with it

1.2 From MAP Inference to Least Squares

27

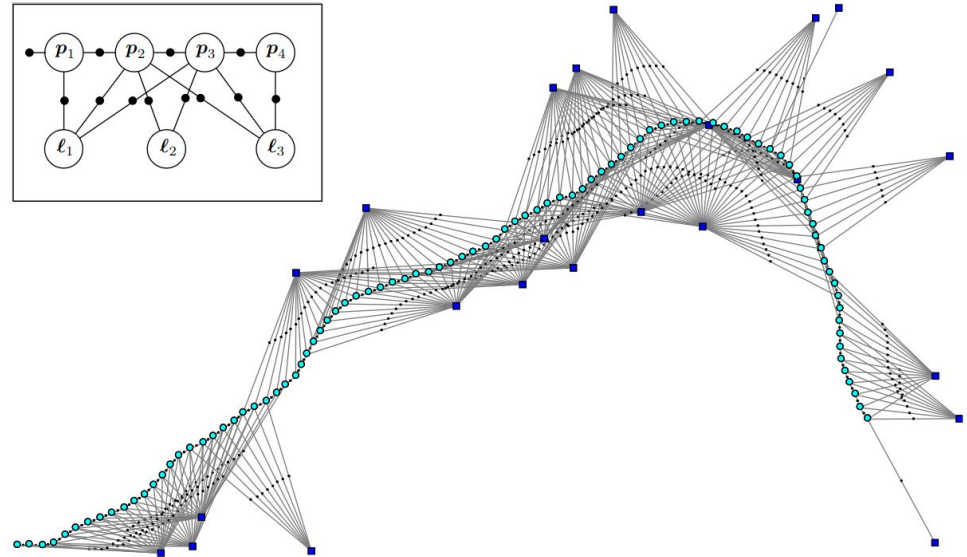


Figure 1.5 Factor graph for a larger, simulated SLAM example.

Factor Graphs for MAP Inference

maximum a posteriori (MAP) estimate

$$\begin{aligned}\mathbf{x}^{\text{MAP}} &= \arg \max_{\mathbf{x}} p(\mathbf{x}|\mathbf{z}) \quad \text{posterior density given} \\ &\quad \text{the measurements} \\ &= \arg \max_{\mathbf{x}} \frac{p(\mathbf{z}|\mathbf{x})p(\mathbf{x})}{p(\mathbf{z})} \quad \text{Bayes' Law} \\ &= \arg \max_{\mathbf{x}} p(\mathbf{z}|\mathbf{x})p(\mathbf{x})\end{aligned}$$

A factor graph F defines the factorization of a global function:

$$\phi(\mathbf{x}) = \prod_i \phi_i(\mathbf{x}_i)$$

For an arbitrary factor graph, MAP inference comes down to maximizing the product of all factor-graph potentials:

$$\begin{aligned}\mathbf{x}^{\text{MAP}} &= \arg \max_{\mathbf{x}} \phi(\mathbf{x}) \\ &= \arg \max_{\mathbf{x}} \prod_i \phi_i(\mathbf{x}_i).\end{aligned}$$



Factor Graphs for MAP Inference

multivariate Gaussian density

$$\mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{|2\pi\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2} \|\boldsymbol{\theta} - \boldsymbol{\mu}\|_{\boldsymbol{\Sigma}}^2\right)$$

squared Mahalanobis distance

$$\|\boldsymbol{\theta} - \boldsymbol{\mu}\|_{\boldsymbol{\Sigma}}^2 \triangleq (\boldsymbol{\theta} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{\theta} - \boldsymbol{\mu})$$

bearing measurement

$$\mathbf{z} = \mathbf{h}(\mathbf{p}, \ell) + \boldsymbol{\eta}$$

motion model

$$\mathbf{g}(\cdot)$$

conditional density of the measurement

$$p(\mathbf{z}|\mathbf{p}, \ell) = \mathcal{N}(\mathbf{z}; \mathbf{h}(\mathbf{p}, \ell), \boldsymbol{\Sigma}_R) = \frac{1}{\sqrt{|2\pi\boldsymbol{\Sigma}_R|}} \exp\left(-\frac{1}{2} \|\mathbf{z} - \mathbf{h}(\mathbf{p}, \ell)\|_{\boldsymbol{\Sigma}_R}^2\right)$$

conditional density of the prior

$$p(\mathbf{p}_{t+1}|\mathbf{p}_t, \mathbf{u}_t) = \frac{1}{\sqrt{|2\pi\boldsymbol{\Sigma}_Q|}} \exp\left(-\frac{1}{2} \|\mathbf{p}_{t+1} - \mathbf{g}(\mathbf{p}_t, \mathbf{u}_t)\|_{\boldsymbol{\Sigma}_Q}^2\right)$$

conditional density of the prior (no control inputs)

$$p(\mathbf{o}_t|\mathbf{p}_{t+1}, \mathbf{p}_t) = \frac{1}{\sqrt{|2\pi\boldsymbol{\Sigma}_S|}} \exp\left(-\frac{1}{2} \|\mathbf{o}_t - (\mathbf{p}_{t+1} - \mathbf{p}_t)\|_{\boldsymbol{\Sigma}_S}^2\right).$$



Factor Graphs for MAP Inference

maximum a posteriori (MAP) estimate

$$\begin{aligned}\mathbf{x}^{\text{MAP}} &= \arg \max_{\mathbf{x}} p(\mathbf{x}|\mathbf{z}) \quad \text{posterior density given the measurements} \\ &= \arg \max_{\mathbf{x}} \frac{p(\mathbf{z}|\mathbf{x})p(\mathbf{x})}{p(\mathbf{z})} \quad \text{Bayes' Law} \\ &= \arg \max_{\mathbf{x}} p(\mathbf{z}|\mathbf{x})p(\mathbf{x})\end{aligned}$$

A factor graph F defines the factorization of a global function:

$$\phi(\mathbf{x}) = \prod_i \phi_i(\mathbf{x}_i)$$

For an arbitrary factor graph, MAP inference comes down to maximizing the product of all factor-graph potentials:

$$\begin{aligned}\mathbf{x}^{\text{MAP}} &= \arg \max_{\mathbf{x}} \phi(\mathbf{x}) \\ &= \arg \max_{\mathbf{x}} \prod_i \phi_i(\mathbf{x}_i).\end{aligned}$$

conditional density of the measurement

$$p(\mathbf{z}|\mathbf{p}, \ell) = \mathcal{N}(\mathbf{z}; \mathbf{h}(\mathbf{p}, \ell), \Sigma_R) = \frac{1}{\sqrt{|2\pi\Sigma_R|}} \exp\left(-\frac{1}{2} \|\mathbf{z} - \mathbf{h}(\mathbf{p}, \ell)\|_{\Sigma_R}^2\right)$$

Let us assume that all factors are proportional to a multivariate Gaussian, i.e.,

$$\phi_i(\mathbf{x}_i) \propto \exp\left(-\frac{1}{2} \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2\right)$$

Taking the negative log and dropping the factor 1/2 allows us to instead minimize a sum of nonlinear least squares terms:

$$\mathbf{x}^{\text{MAP}} = \arg \min_{\mathbf{x}} \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2$$



Solving Linear Least Squares

nonlinear least squares

$$\mathbf{x}^{\text{MAP}} = \arg \min_{\mathbf{x}} \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2$$

Taylor expansion

$$\mathbf{h}_i(\mathbf{x}_i) = \mathbf{h}_i(\mathbf{x}_i^0 + \delta_i) \approx \mathbf{h}_i(\mathbf{x}_i^0) + \mathbf{H}_i \delta_i$$

$$\mathbf{H}_i \triangleq \left. \frac{\partial \mathbf{h}_i(\mathbf{x}_i)}{\partial \mathbf{x}_i} \right|_{\mathbf{x}_i^0}$$

linear least squares

$$\begin{aligned} \delta^* &= \arg \min_{\delta} \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0) - \mathbf{H}_i \delta_i\|_{\Sigma_i}^2 \\ &= \arg \min_{\delta} \sum_i \|(\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0)) - \mathbf{H}_i \delta_i\|_{\Sigma_i}^2 \end{aligned}$$

rewrite the square Mahalanobis norm

$$\|e\|_{\Sigma}^2 \triangleq e^{\top} \Sigma^{-1} e = \left(\Sigma^{-1/2} e \right)^{\top} \left(\Sigma^{-1/2} e \right) = \left\| \Sigma^{-1/2} e \right\|_2^2$$

whitening

$$\mathbf{A}_i = \Sigma_i^{-1/2} \mathbf{H}_i$$

$$\mathbf{b}_i = \Sigma_i^{-1/2} (\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0))$$

linear least squares

$$\begin{aligned} \delta^* &= \arg \min_{\delta} \sum_i \|\mathbf{A}_i \delta_i - \mathbf{b}_i\|_2^2 \\ &= \arg \min_{\delta} \|\mathbf{A} \delta - \mathbf{b}\|_2^2, \end{aligned}$$

normal equations

$$(\mathbf{A}^{\top} \mathbf{A}) \delta^* = \mathbf{A}^{\top} \mathbf{b}.$$



Factor Graphs and Sparsity

$$[A|b] = \begin{array}{c} \phi_1 \\ \phi_2 \\ \phi_3 \\ \phi_4 \\ \phi_5 \\ \phi_6 \\ \phi_7 \\ \phi_8 \\ \phi_9 \end{array} \left[\begin{array}{cccccc} \delta \ell_1 & \delta \ell_2 & \delta p_1 & \delta p_2 & \delta p_3 & b \\ & & A_{13} & & & b_1 \\ & & A_{23} & A_{24} & & b_2 \\ & & & A_{34} & A_{35} & b_3 \\ A_{41} & & & & & b_4 \\ & A_{52} & & & & b_5 \\ & & A_{63} & & & b_6 \\ A_{71} & & A_{73} & & & b_7 \\ A_{81} & & & A_{84} & & b_8 \\ & A_{92} & & & A_{95} & b_9 \end{array} \right]$$

$$\Lambda \triangleq A^\top A = \begin{bmatrix} \Lambda_{11} & & \Lambda_{13} & \Lambda_{14} & \\ & \Lambda_{22} & & & \Lambda_{25} \\ \Lambda_{31} & & \Lambda_{33} & \Lambda_{34} & \\ \Lambda_{41} & & \Lambda_{43} & \Lambda_{44} & \Lambda_{45} \\ & \Lambda_{52} & & \Lambda_{54} & \Lambda_{55} \end{bmatrix}$$

(a) Information matrix $\Lambda \triangleq A^\top A$.

1.1 Visualizing SLAM With Factor Graphs

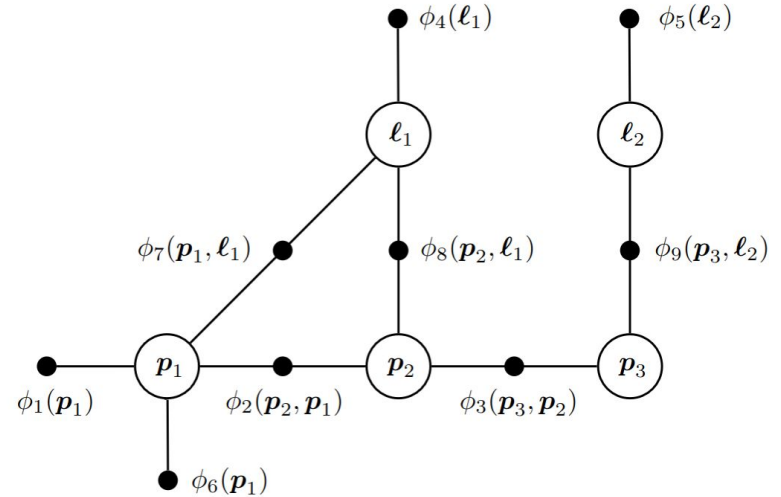
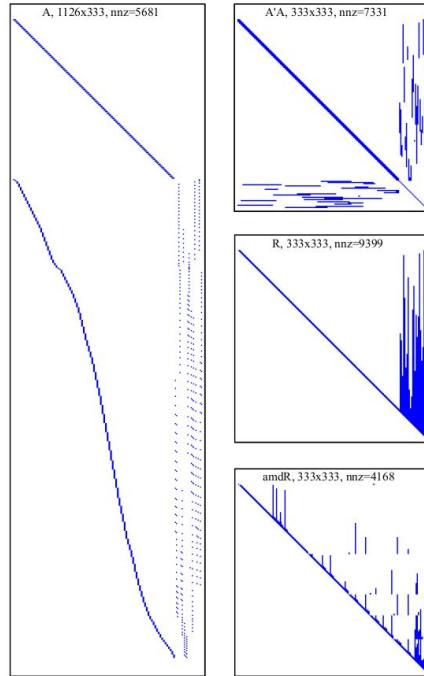


Figure 1.3 Factor graph resulting from the example in Figure 1.1.



Factor Graphs and Sparsity



1.2 From MAP Inference to Least Squares

27

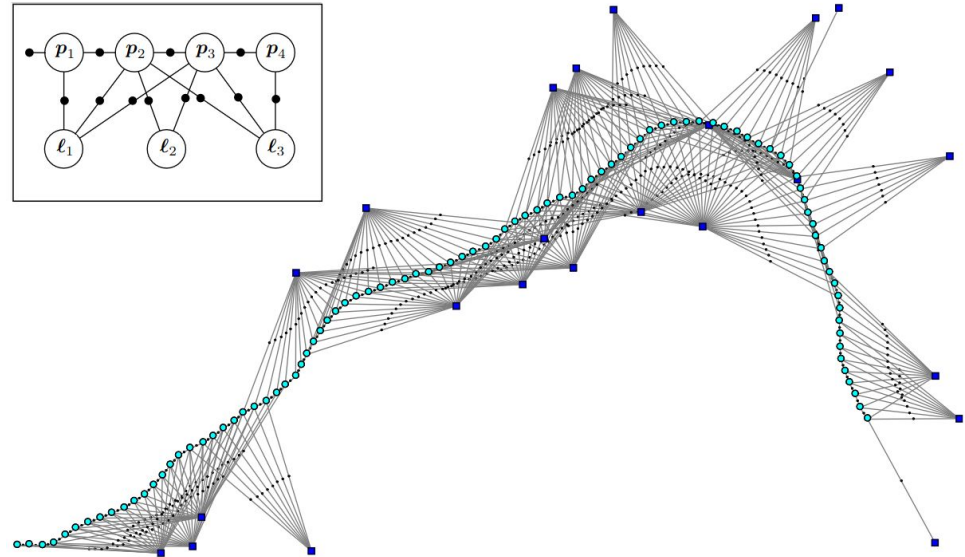
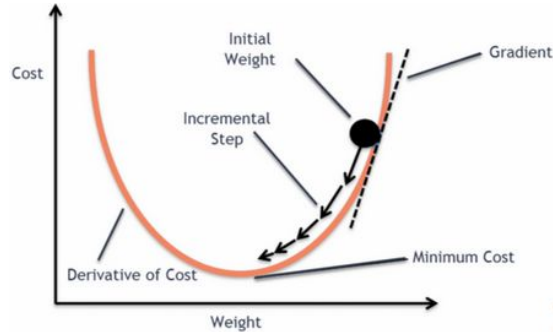


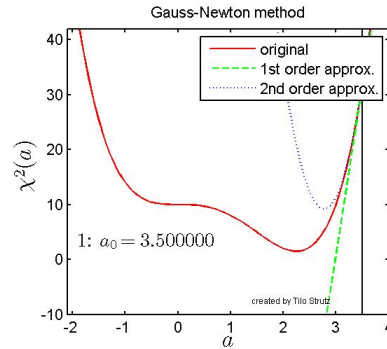
Figure 1.5 Factor graph for a larger, simulated SLAM example.

Nonlinear Optimization

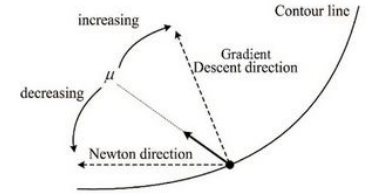
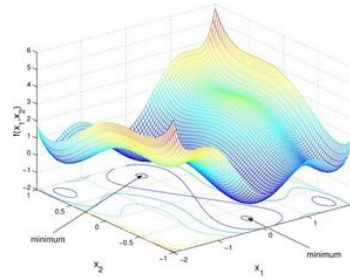
Steepest Descent



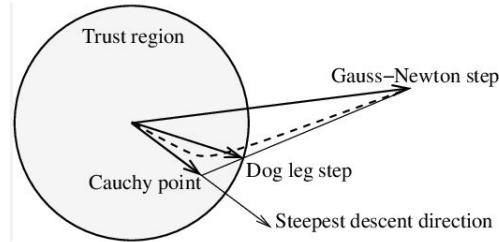
Gauss Newton



Levenberg-Marquardt



Dogleg Minimization



Solvers

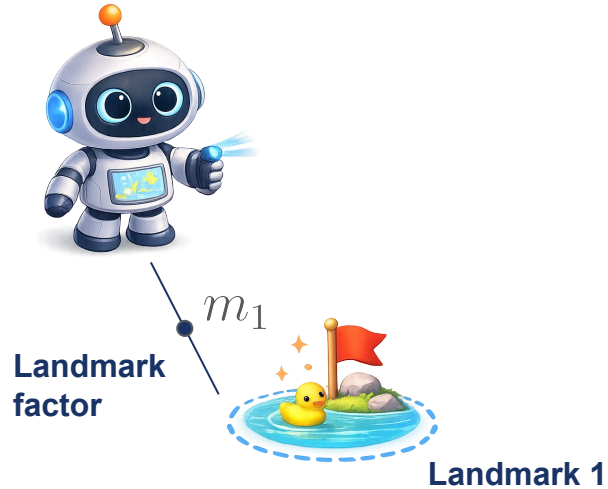
- ❖ GTSAM <https://gtsam.org/>
- ❖ G2o <https://openslam-org.github.io/g2o.html>
- ❖ Ceres <https://github.com/ceres-solver/ceres-solver>



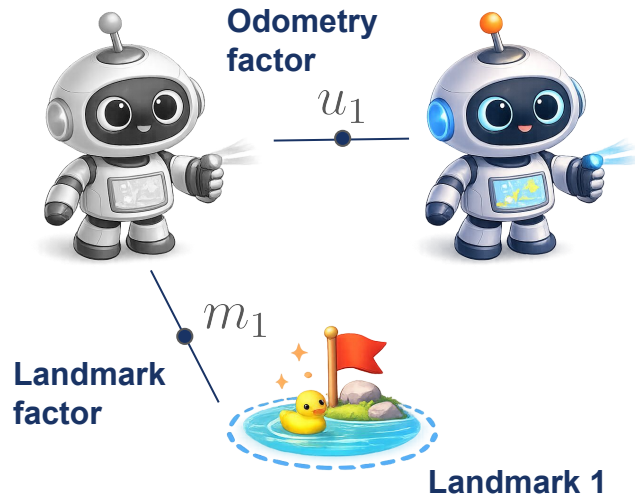
The SLAM problem



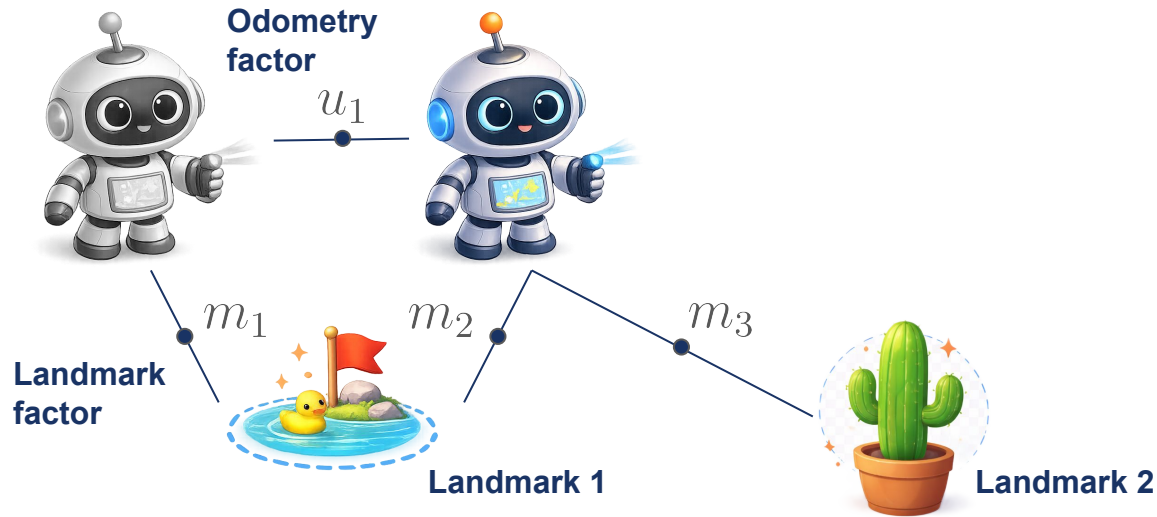
The SLAM problem ($t = 0$)



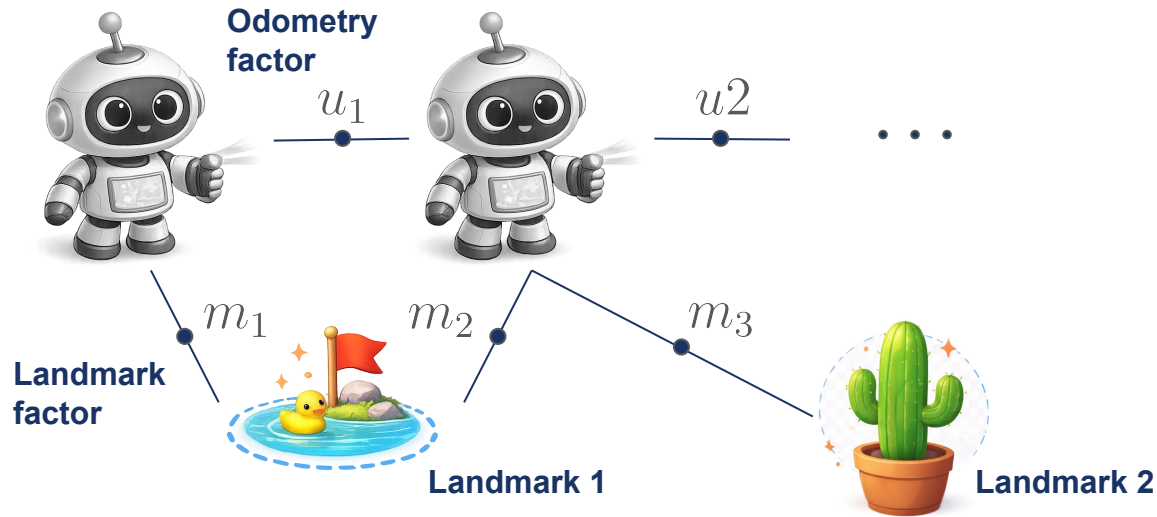
The SLAM problem ($t = 1$)



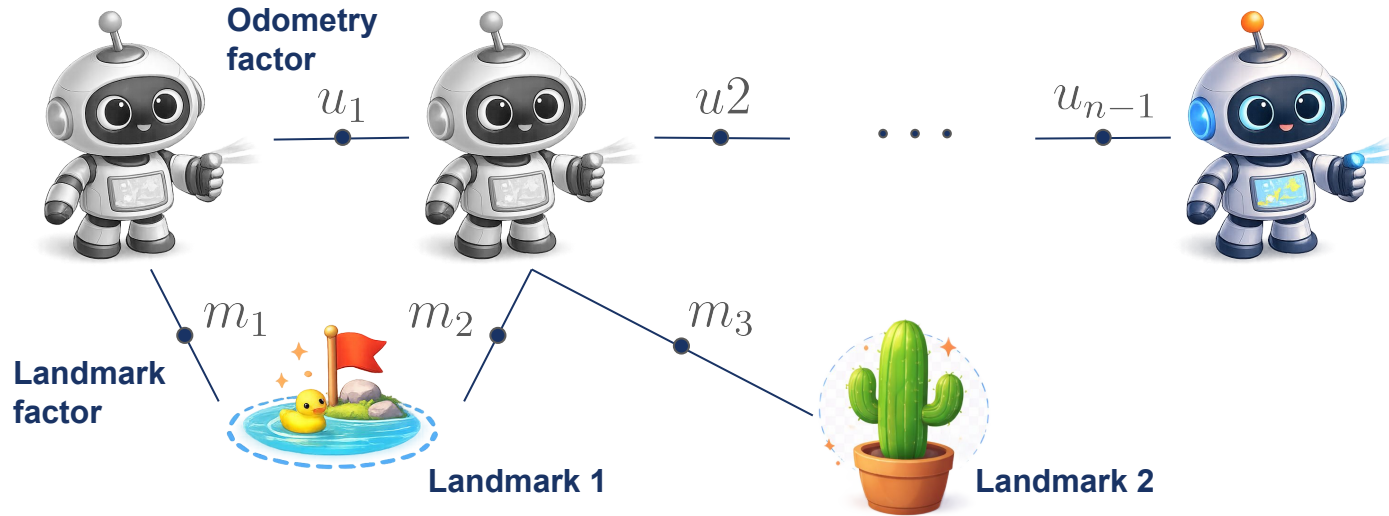
The SLAM problem ($t = 1$)



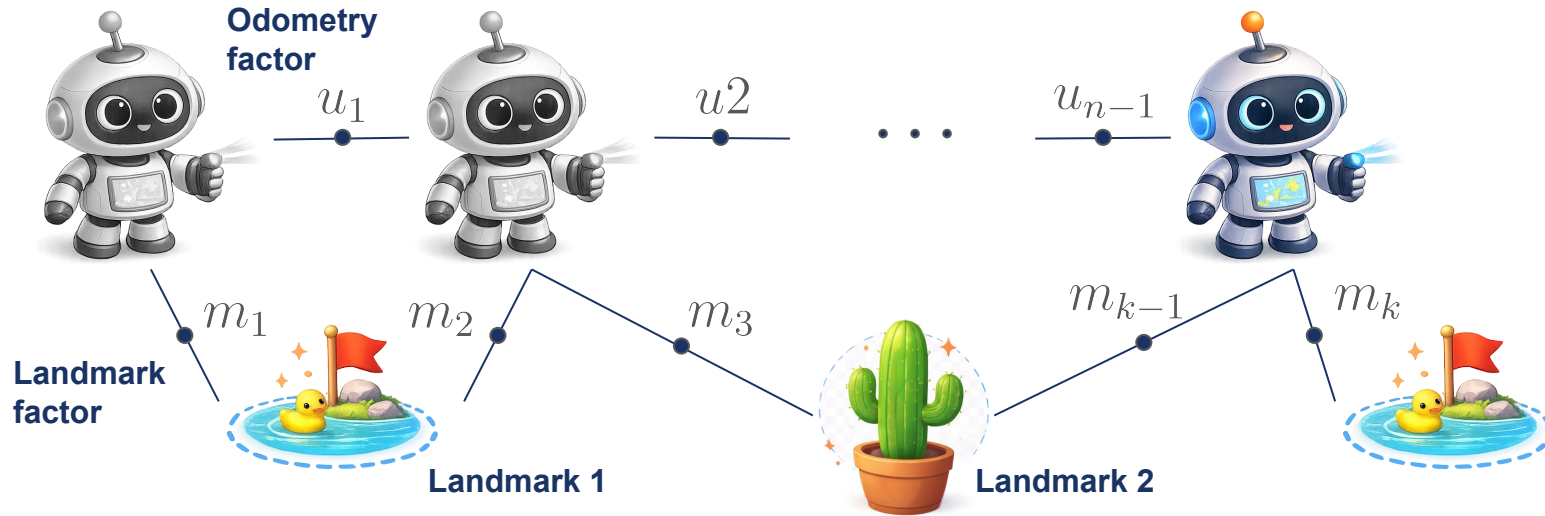
The SLAM problem ($t = \dots$)



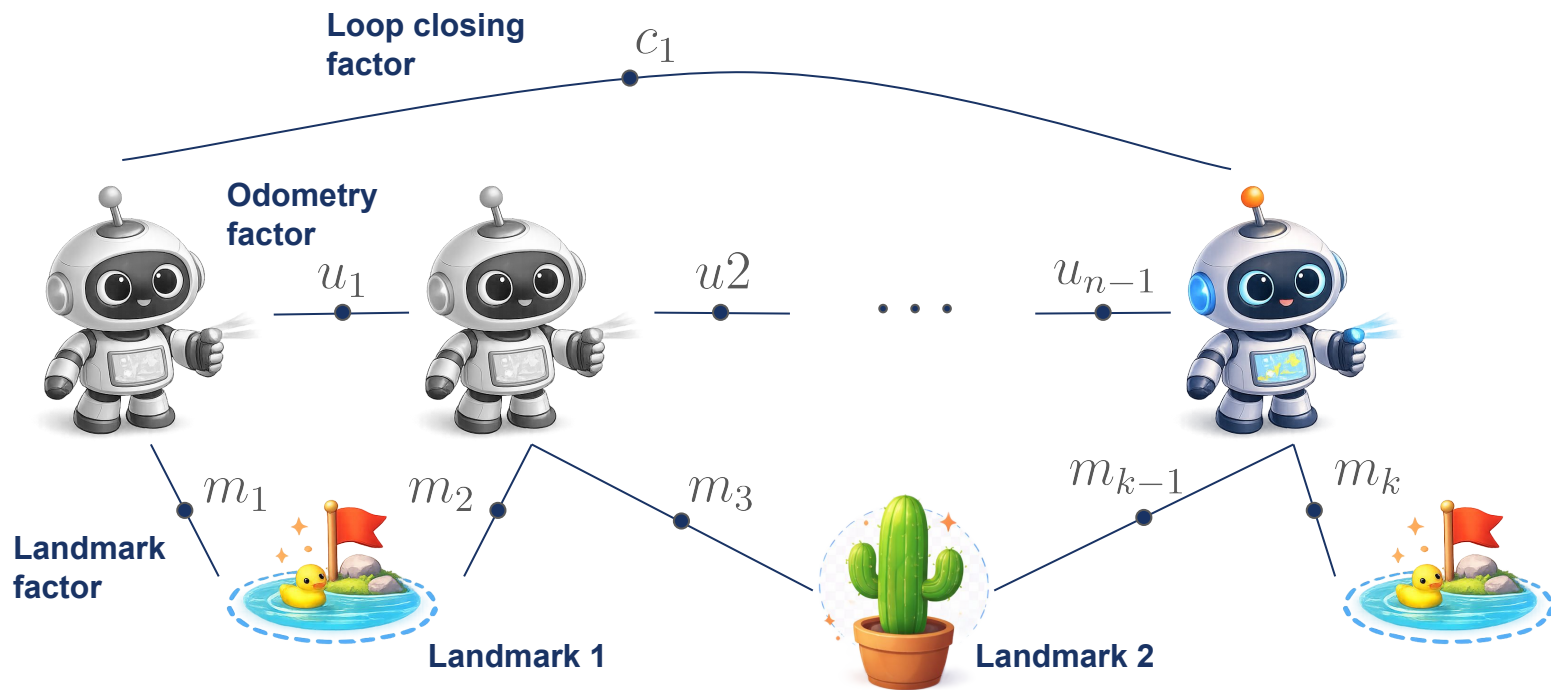
The SLAM problem ($t = n-1$)



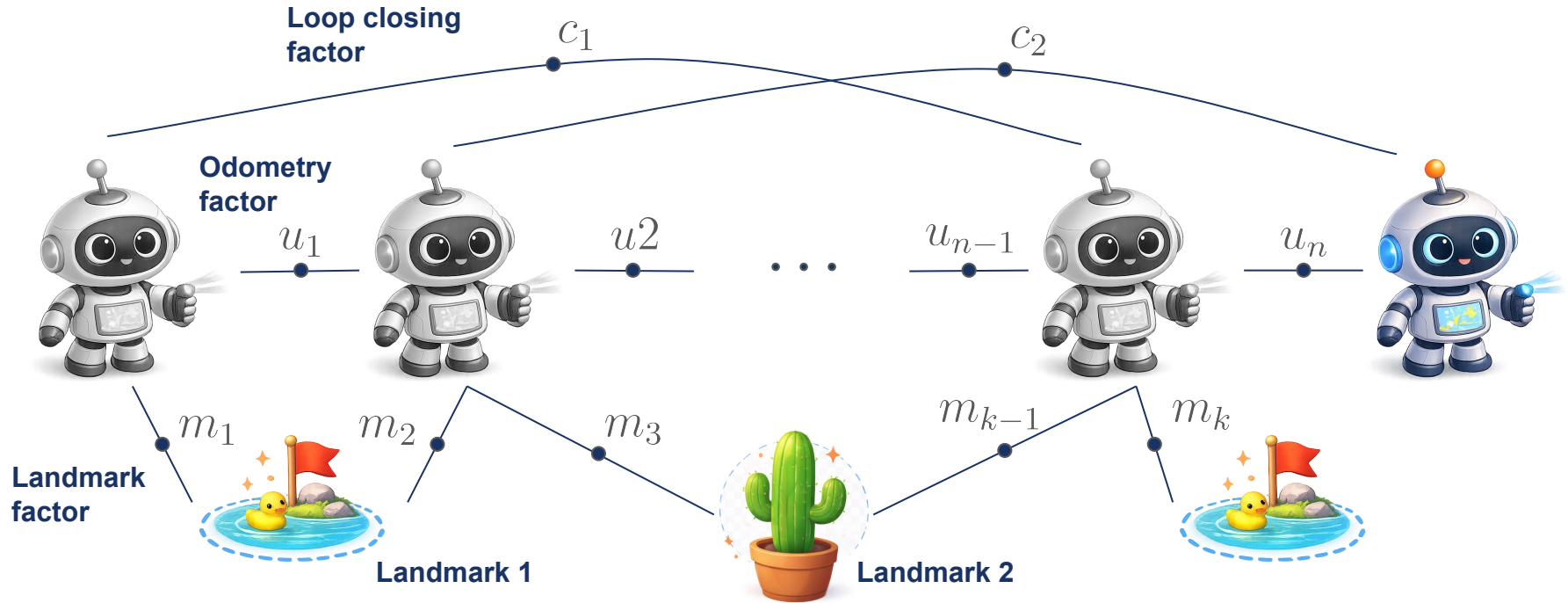
The SLAM problem ($t = n-1$)



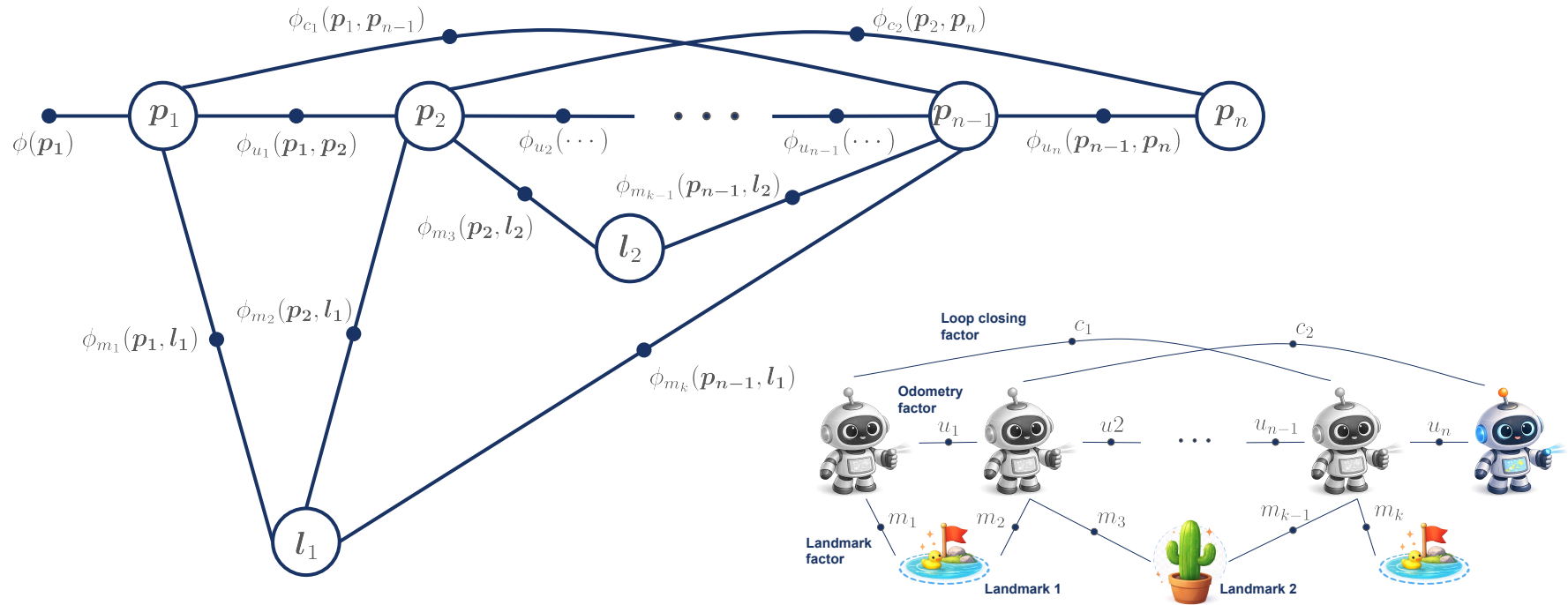
The SLAM problem ($t = n-1$)



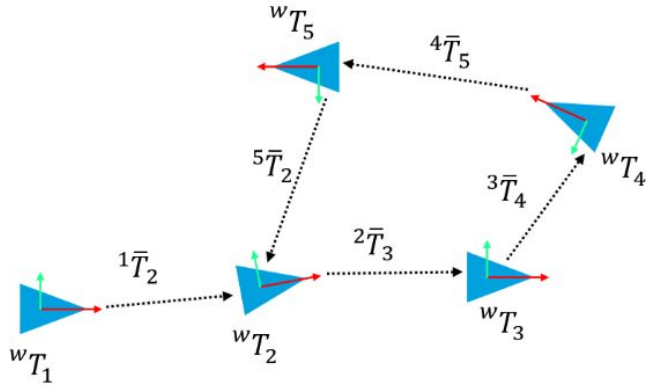
The SLAM problem ($t = n$)



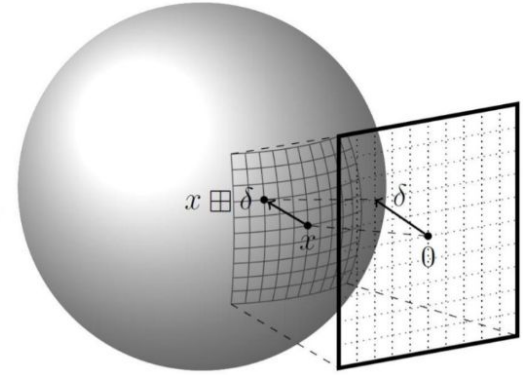
The SLAM problem (t = n)



Pose Graph Optimization: Example



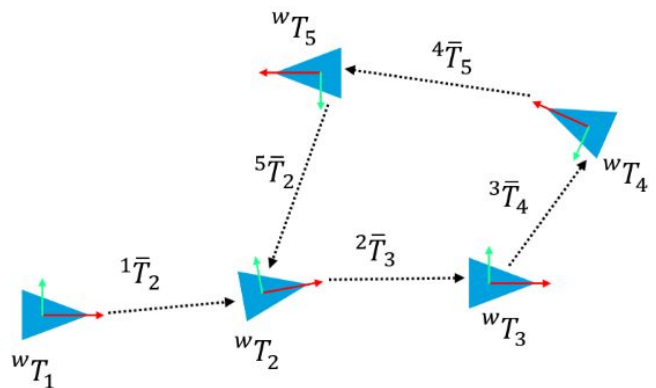
$$\begin{aligned} \boxplus &: \mathcal{S} \times \mathbb{R}^n \rightarrow \mathcal{S}, \\ \boxminus &: \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}^n \end{aligned}$$



$$\min_{\mathbf{T}_t, t=1, \dots, n} \sum_{(i,j) \in \mathcal{E}} \|(\mathbf{T}_i^{-1} \mathbf{T}_j) \boxminus \bar{\mathbf{T}}_j^i\|_{\Sigma_{ij}}^2$$

Note: $T = \begin{bmatrix} R & t \\ \mathbf{0} & 1 \end{bmatrix} \in \text{SE}(3)$ is the homogeneous transformation matrix

Pose Graph Optimization: Example



$$\min_{\mathbf{T}_t, t=1, \dots, n} \sum_{(i,j) \in \mathcal{E}} \|(\mathbf{T}_i^{-1} \mathbf{T}_j) \ominus \bar{\mathbf{T}}_j^i\|_{\Sigma_{ij}}^2$$

Jacobian $\mathbf{J}$

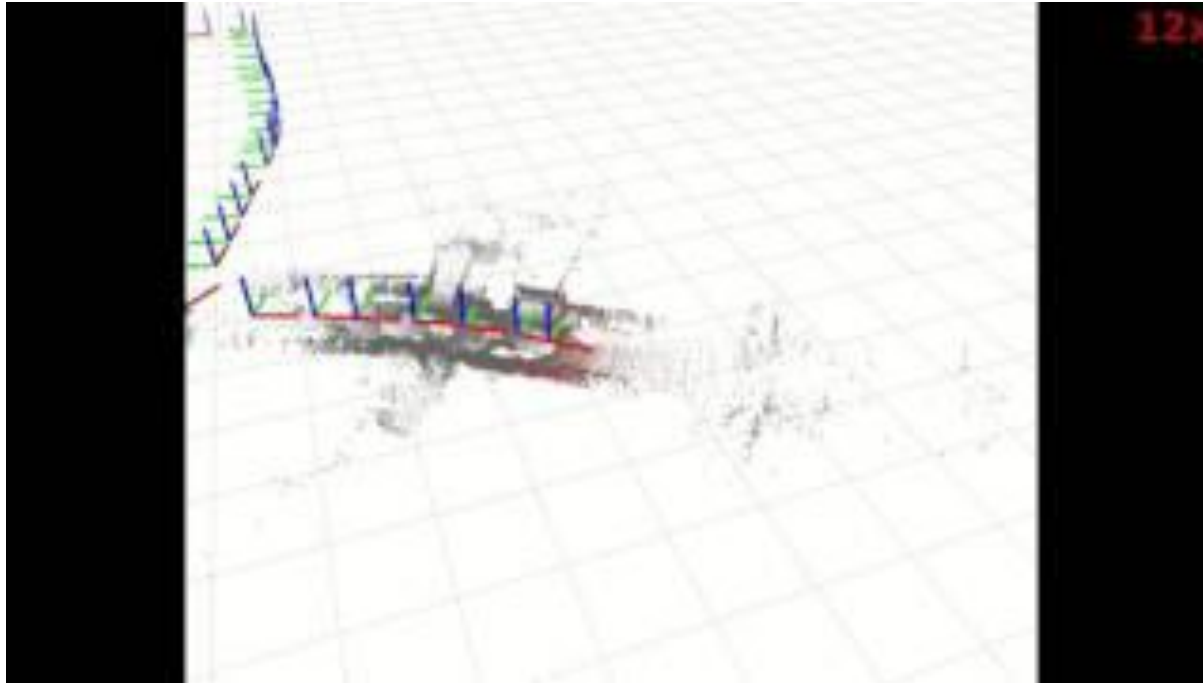
	T_1	T_2	T_3	T_4	T_5	T_6	T_7
$\bar{T}_{12}$	■	■					
$\bar{T}_{23}$		■	■				
$\bar{T}_{34}$			■	■			
$\bar{T}_{45}$				■	■		
$\bar{T}_{56}$					■	■	
$\bar{T}_{67}$						■	■
$\bar{T}_{52}$		■			■		

Hessian $\mathbf{J}^T \mathbf{J}$

	T_1	T_2	T_3	T_4	T_5	T_6	T_7
T_1	■	■					
T_2	■	■	■		■		
T_3		■	■	■	■		
T_4			■	■	■		
T_5		■		■	■	■	
T_6				■	■	■	■
T_7					■	■	■

$$\min_{w_{T_1}, \dots, w_{T_5} \in \text{SE}(3)} \|({}^wT_1^{-1} {}^wT_2) - {}^1\bar{T}_2\| + \|({}^wT_2^{-1} {}^wT_3) - {}^2\bar{T}_3\| + \dots \|({}^wT_5^{-1} {}^wT_2) - {}^5\bar{T}_2\|$$

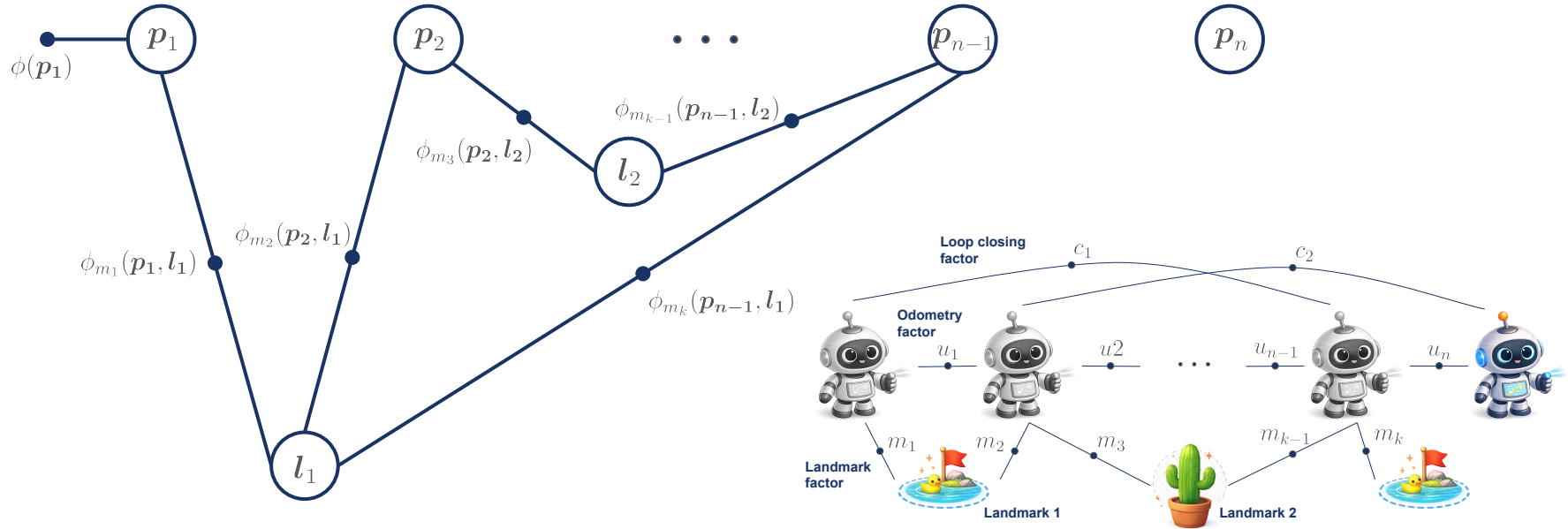
Pose Graph Optimization: Example



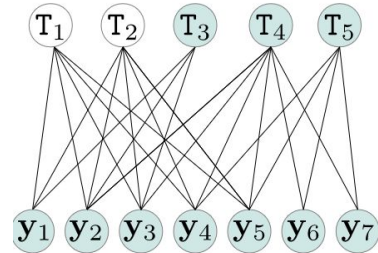
https://youtu.be/KYvOqUB_odg



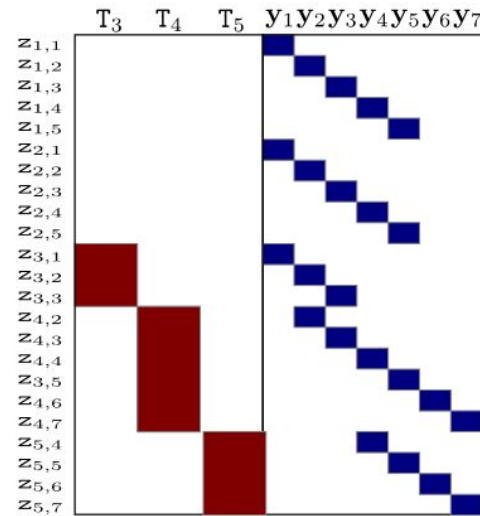
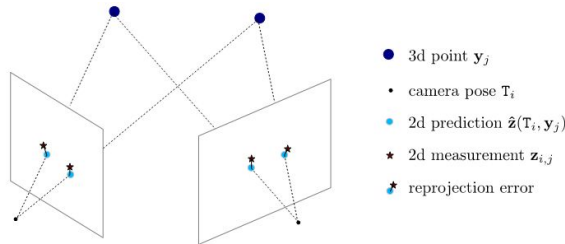
Landmark-based SLAM



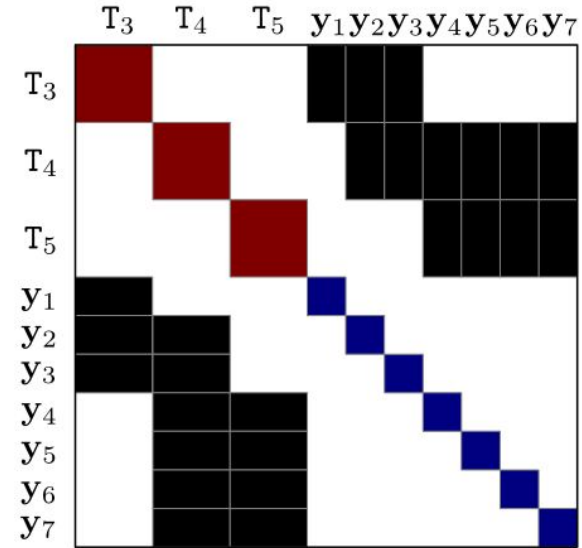
Landmark-based SLAM



(a) Markov random field



(b) Jacobian



(c) Hessian



Robustness to Outliers: Robust loss functions

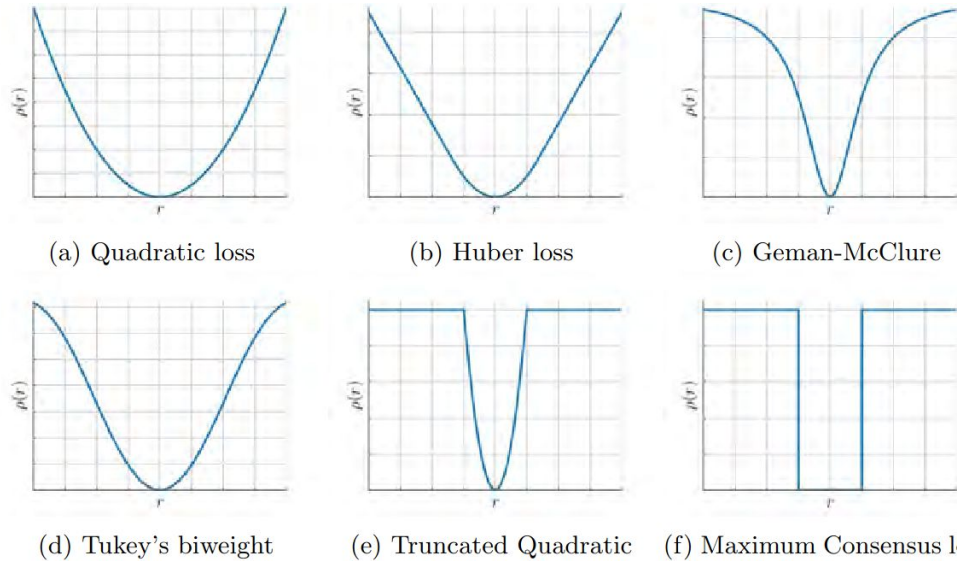


Figure 3.5 Quadratic loss and examples of robust loss functions. The shape of the robust loss functions is controlled by a parameter that controls the separation between inliers and outliers.





Centre for
Robotics



SLAM Course

08 - Factor Graphs

Dr Alejandro Fontán Villacampa
April 2026